

Sistema de Seguimiento de Hábitos Saludables

Plan de Proyecto — Releases y Sprints

Información General

Campo	Detalle
Proyecto	Sistema de Seguimiento de Hábitos Saludables
Tipo	Proyecto de Aula — Nivel Básico
Stack	NestJS + Next.js + PostgreSQL + Docker + Prisma
Entidades	Usuario, CategoriaHabito, Habito, Meta, RegistroDiario, Racha
Equipo	Juan José Horta Vanegas Javier Esteban Ortiz López

Plan de Releases

Release 1 — Segundo Corte: Backend Base + Autenticación + Hábitos

Objetivo: Infraestructura Docker + Prisma, módulos CRUD de Usuario (con JWT), CategoriaHabito, Habito y Meta. Common Module. Frontend con vistas base.

Sprint	Período	Festivos	Alcance	HUs
Sprint 1 Infraestructura y Usuarios	Mar 16 → Mar 29	Mar 23 (San José)	Docker Compose, Prisma schema, migraciones. CRUD de Usuario + Auth JWT (registro/login). CRUD de CategoriaHabito.	HU-01 HU-02
Sprint 2 Hábitos, Metas y Common	Mar 30 → Abr 10	Abr 2 (J. Santo) Abr 3 (V. Santo)	CRUD de Habito con relación a Categoría y Usuario. CRUD de Meta personal por hábito. Common Module (Filters, Pipes, Interceptors).	HU-03 HU-04 HU-05
Sprint 3 Registros y Frontend base	Abr 13 → Abr 17	—	Módulo RegistroDiario (cumplimiento diario + observaciones). Frontend: Next.js, listados de hábitos, formularios de registro.	HU-06 HU-07

Cierre Segundo Corte: 17 de Abril de 2026

Release 2 — Tercer Corte: Racha, Estadísticas e Integración Completa

Objetivo: Módulo de Racha, resumen semanal/mensual, reporte de progreso, integración completa frontend ↔ backend. Despliegue funcional con Docker.

Sprint	Período	Festivos	Alcance	HUs
--------	---------	----------	---------	-----

Sprint 4 Frontend avanzado e integración	Abr 20 → May 8	May 1 (Día del Trabajo)	Dashboard personal con resumen de hábitos. Página de registro diario con selects dinámicos. Navegación, layout y vistas de metas.	HU-08 HU-09
Sprint 5 Racha, reportes y cierre	May 11 → May 22	May 18 (Ascensión)	Módulo Racha (días consecutivos + racha máxima). Historial mensual y reporte por rango de fechas. Pruebas E2E + Docker compose validación final.	HU-10

Cierre Tercer Corte: 22 de Mayo de 2026

Festivos Colombianos 2026 (Marzo — Mayo)

Fecha	Festivo	Sprint afectado
Lunes 23 de Marzo	Día de San José	Sprint 1
Jueves 2 de Abril	Jueves Santo	Sprint 2
Viernes 3 de Abril	Viernes Santo	Sprint 2
Viernes 1 de Mayo	Día del Trabajo	Sprint 4
Lunes 18 de Mayo	Día de la Ascensión	Sprint 5

Historias de Usuario

HU-01 — Gestión de Usuarios		Sprint 1 CU-01
Historia de Usuario		
Como administrador del sistema, quiero registrar, consultar, editar y eliminar usuarios con sus datos personales (edad, peso, estatura), para mantener un registro centralizado de los clientes del centro de bienestar.		
Criterios de Aceptación ■ Crear usuario con: nombres, apellidos, correo, contraseña, edad, peso y estatura. ■ El correo es único; duplicados retornan error claro. ■ Se puede consultar la lista de usuarios con paginación. ■ Se puede consultar un usuario por su ID con datos completos. ■ Se puede editar la información de un usuario existente. ■ Se puede eliminar un usuario que no tenga hábitos activos; si los tiene, retorna error descriptivo.	Tareas Técnicas • Backend: CreateUsuarioDto, UpdateUsuarioDto con class-validator • Backend: UsuarioRepository (Prisma queries) • Backend: UsuarioService (lógica + validaciones) • Backend: UsuarioController (GET, GET/:id, POST, PUT/:id, DELETE/:id) • Frontend: Página /usuarios y formulario /usuarios/new • Frontend: Página de detalle /usuarios/[id]	
HU-02 — Autenticación de Usuarios (JWT)		Sprint 1 CU-01
Historia de Usuario		
Como usuario del sistema, quiero registrarme e iniciar sesión con correo y contraseña mediante JWT, para acceder de forma segura a mis hábitos y registros personales.		
Criterios de Aceptación ■ Endpoint POST /auth/register crea usuario y retorna token JWT. ■ Endpoint POST /auth/login valida credenciales y retorna token JWT. ■ Contraseñas almacenadas con hash bcrypt (nunca en texto plano). ■ Rutas protegidas retornan 401 si no se envía token válido. ■ El token incluye el ID del usuario para identificar registros propios.	Tareas Técnicas • Backend: AuthModule con JwtModule y PassportModule • Backend: AuthService (register, login, validateUser) • Backend: JwtStrategy + AuthGuard para rutas protegidas • Frontend: Páginas /auth/register y /auth/login • Frontend: Almacenamiento del token y manejo de sesión	

Historia de Usuario

Como administrador, **quiero** crear y gestionar categorías de hábitos (hidratación, ejercicio, alimentación, sueño, meditación), **para** organizar los hábitos por tipo y facilitar la navegación del usuario.

Criterios de Aceptación

- Crear categoría con nombre y descripción.
- El nombre de la categoría es único; duplicados retornan error.
- Listar todas las categorías disponibles.
- Editar una categoría existente.
- Eliminar una categoría sin hábitos asociados; si tiene hábitos, retorna error.

Tareas Técnicas

- Backend: DTOs, Repository, Service, Controller del módulo CategoriaHbito
- Frontend: Listado y formulario de categorías

HU-04 — Gestión de Hábitos

Sprint 2 | CU-02

Historia de Usuario

Como usuario autenticado, **quiero** crear y gestionar mis hábitos personalizados asociados a una categoría, **para** definir qué hábitos quiero seguir y organizarlos por tipo.

Criterios de Aceptación

- Crear hábito con: nombre, descripción, categoría asociada y usuario propietario.
- El usuario solo puede ver y gestionar sus propios hábitos.
- La categoría referenciada debe existir; si no, retorna error.
- Listar hábitos con filtro opcional por categoría.
- Editar y eliminar hábitos sin registros diarios asociados.

Tareas Técnicas

- Backend: HabitoRepository, HabitoService, HabitoController
- Backend: DTO con validación de FK (categoriaHabitId, usuariId)
- Frontend: Listado con filtro por categoría, formulario con select de categoría
- Frontend: Página de detalle /habitos/[id]

HU-05 — Gestión de Metas Personales

Sprint 2 | CU-03

Historia de Usuario

Como usuario autenticado, **quiero** definir una meta personal para cada hábito (ej: beber 8 vasos de agua al día), **para** establecer un objetivo medible que sirva como referencia para el seguimiento.

Criterios de Aceptación

- Crear meta con: hábito asociado, descripción del objetivo y valor meta numérico.
- Un hábito solo puede tener una meta activa a la vez.
- Se puede editar y desactivar una meta existente.
- Listar metas activas del usuario autenticado.
- Si no existe meta para un hábito, el sistema muestra aviso invitando a crearla.

Tareas Técnicas

- Backend: MetaRepository con validación de unicidad activa por hábito
- Backend: MetaService, MetaController (GET, POST, PUT/:id)
- Frontend: Formulario de meta con select de hábito
- Frontend: Listado de metas activas con opción de edición

HU-06 — Common Module: Filtros, Interceptores y Pipes

Sprint 2

Historia de Usuario

Como desarrollador, **quiero** implementar un módulo compartido con filtros de excepción, interceptores de respuesta y pipes de validación, **para** garantizar respuestas consistentes y manejo centralizado de errores en toda la API.

Criterios de Aceptación

- Todas las excepciones HTTP retornan JSON con formato: { statusCode, message, error, timestamp }.
- Todas las respuestas exitosas retornan: { statusCode, message, data }.
- ValidationPipe global configurado con whitelist: true y forbidNonWhitelisted: true.
- Errores de validación indican qué campo falló y por qué.

Tareas Técnicas

- common/filters/http-exception.filter.ts
- common/interceptors/response.interceptor.ts
- common/pipes/validation.pipe.ts
- Registro global en main.ts

HU-07 — Registro Diario de Cumplimiento

Sprint 3 | CU-04, CU-05, CU-07

Historia de Usuario

Como usuario autenticado, **quiero** registrar diariamente el cumplimiento de un hábito con valor y observaciones, **para** llevar un control preciso de mi progreso día a día.

Criterios de Aceptación

- Crear registro con: hábito, fecha, valor registrado y observaciones opcionales (CU-04).
- No se permite duplicar el registro del mismo hábito en la misma fecha.
- El valor registrado se compara automáticamente con la meta del hábito.
- Editar o eliminar un registro erróneo (CU-07).
- Listar registros de un hábito con filtro por rango de fechas.
- Consultar resumen semanal de cumplimiento por hábito (CU-05).

Tareas Técnicas

- Backend: RegistroDiarioRepository con unicidad (habitoid + fecha)
- Backend: RegistroDiarioService, RegistroDiarioController
- Backend: Endpoint GET /resumen/semanal con query param de fecha
- Frontend: Formulario de registro diario con select de hábito
- Frontend: Listado con indicador de cumplimiento vs. meta

HU-08 — Visualización de Racha Actual

Sprint 4 | CU-06

Historia de Usuario

Como usuario autenticado, **quiero** ver la racha actual (días consecutivos cumpliendo un hábito) y la racha máxima histórica, **para** mantener la motivación al visualizar mi progreso continuo (CU-06).

Criterios de Aceptación

- El sistema calcula automáticamente los días consecutivos de cumplimiento.
- La racha se rompe si hay un día sin registro o con incumplimiento.
- Se muestra la racha actual y la racha máxima histórica por hábito.
- La racha se actualiza automáticamente al registrar o editar un RegistroDiario.
- La entidad Racha almacena: habitoid, rachaActual, rachaMaxima, ultimaFecha.

Tareas Técnicas

- Backend: Entidad Racha en Prisma schema
- Backend: Lógica de actualización de racha en RegistroDiarioService
- Backend: Endpoint GET /habitos/:id/racha
- Frontend: Indicador visual de racha en la vista de cada hábito

HU-09 — Frontend: Dashboard, Navegación y Layout General

Sprint 4 | CU-09

Historia de Usuario

Como usuario del sistema, **quiero** navegar entre las secciones del sistema y ver mi dashboard personal desde la interfaz web, **para** gestionar el seguimiento de hábitos sin depender de herramientas externas como Postman.

Criterios de Aceptación

- Layout con sidebar/navbar con enlaces: Hábitos, Categorías, Metas, Registros, Estadísticas.
- Navegación indica visualmente la sección activa.
- Dashboard personal con resumen: hábitos activos, % cumplimiento semanal y racha actual.
- Página de registro diario con selects dinámicos (hábito → fecha → valor).
- Formularios con mensajes de error claros y estados de carga (spinner/skeleton).
- Diseño responsivo funcional en desktop y móvil.

Tareas Técnicas

- Frontend: layout.tsx con Sidebar/Navbar y rutas activas
- Frontend: Página /dashboard con tarjetas de hábito y barras de progreso
- Frontend: Página /registro/new con selects encadenados
- Frontend: Componentes de feedback (toast/alert de éxito/error)
- Frontend: Estilos responsivos con Tailwind CSS

Historia de Usuario

Como usuario o profesional del centro de bienestar, **quiero** consultar el historial mensual de cumplimiento y generar un reporte de progreso por rango de fechas, **para** evaluar el rendimiento a largo plazo y monitorear el avance de los pacientes (CU-08, CU-10).

Criterios de Aceptación

- Endpoint que retorna historial mensual de un hábito agrupado por mes.
- Cada mes muestra: días registrados, días cumplidos y porcentaje de cumplimiento.
- Reporte por rango de fechas: seleccionar fecha inicio y fin.
- Reporte muestra por hábito: días cumplidos, % cumplimiento y racha máxima en el período.
- Vista de profesional puede filtrar por usuario (CU-09 — listar usuarios y metas activas).
- Pruebas E2E del flujo completo y validación Docker compose final.

Tareas Técnicas

- Backend: GET /historial/mensual con query param (habitoid)
- Backend: GET /reportes/progreso con params (usuarioid, fechaInicio, fechaFin)
- Frontend: Página /historial con tabla agrupada por mes
- Frontend: Página /reportes/progreso con filtros de fecha y tabla de resultados
- Backend: Pruebas E2E con Jest + Supertest
- Infraestructura: docker-compose.yml validado en entorno limpio

Modelo de Datos (Prisma)

Relación	Cardinalidad
Usuario  Habito	1  N
Usuario  RegistroDiario	1  N
CategoriaHabito  Habito	1  N
Habito  Meta	1  N
Habito  RegistroDiario	1  N
Habito  Racha	1  1

Resumen de Entidades

Entidad	Campos principales
Usuario	id, nombres, apellidos, correo (unique), contraseña, edad, peso, estatura
CategoriaHabito	id, nombre (unique), descripcion
Habito	id, nombre, descripcion, categoriaHabitoId, usuarioid
Meta	id, habitoid, descripcion, valorMeta, activa
RegistroDiario	id, habitoid, usuarioid, fecha, valorRegistrado, observaciones [unique: habitoid+fecha]
Racha	id, habitoid (unique), rachaActual, rachaMaxima, ultimaFecha

Definition of Done (DoD) Global

Backend

- Endpoint(s) implementados con arquitectura en capas: Controller → Service → Repository.
- DTOs con validaciones usando class-validator y class-transformer.
- Manejo de errores con excepciones HTTP apropiadas (NotFoundException, ConflictException, BadRequestException).
- Respuestas con formato uniforme (interceptor aplicado).
- Endpoint probado manualmente con Postman/Thunder Client y funcionando correctamente.

Frontend

- Página(s) implementada(s) con componentes reutilizables.
- Consumo del API a través de la capa de services/.
- Manejo de estados: carga (loading), éxito y error.
- Formularios con validación del lado del cliente.
- Diseño responsivo y navegable.

Infraestructura y Código

- Código versionado en GitHub con commits descriptivos.
- El servicio funciona correctamente con docker compose up.
- No hay errores de consola ni advertencias críticas.
- Las migraciones de Prisma están aplicadas y el esquema es consistente.

Estado del Proyecto

- Plan de releases revisado y aprobado
- Historias de Usuario revisadas y aprobadas (10 HUs)
- Criterios de Aceptación revisados y aprobados
- Definition of Done revisado y aprobado
- Modelo de datos revisado y aprobado
- Repositorio GitHub creado con Issues y Milestones
- Sprint 1 iniciado